

ANNAI MIRA COLLEGE OF ENGINEERING AND TECHNOLOGY

NH-46, Chennai-Bengaluru National Highways, Arappakkam, Ranipet- 632517, Tamil Nadu, India

Telephone : 04172-292925 / Fax : 04172-292926

Email : amcet.rtet@gmail.com/info@amcet.in Web: www.amcet.in

DEPARTMENT OF INFORMATION TECHNOLOGY



CCS365 – SOFTWARE DEFINED NETWORKS LABORATORY

NAME :

REGISTER NUMBER :

YEAR & BRANCH :

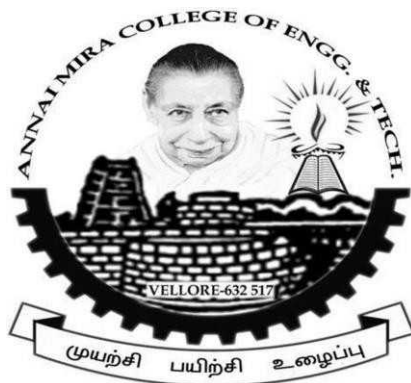
SEMESTER :

ACADEMIC YEAR :

ANNAI MIRA COLLEGE OF ENGINEERING AND TECHNOLOGY

NH-46, Chennai-Bengaluru National Highways, Arappakkam, Ranipet - 632517, TamilNadu, India

Telephone : 04172-292925 / Fax : 04172-292926



CERTIFICATE

This is to Certify that the Bonafide record of the practical work done by Register Number of III year **B.TECH INFORMATION TECHNOLOGY** submitted for the B.TECH - Degree practical examination (VI Semester) in **CCS365 – SOFTWARE DEFINED NETWORKS LABORATORY** during the academic year **2025 –2026**.

STAFF IN-CHARGE

HEAD OF THE DEPARTMENT

Submitted for the practical examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENT

SI.NO	DATE	NAME OF THE EXPERIMENTS	PAGE NO	MARKS	SIGN
1		Setup your own virtual SDN lab i)VirtualBox/Mininet Environment for SDN - http://mininet.org ii) https://www.kathara.org iii) GNS3			
2		Create a simple mininet topology with SDN controller and use Wireshark to capture and visualize the OpenFlow messages such as OpenFlow FLOW MOD, PACKET IN, PACKET OUT etc.			
3		Create a SDN application that uses the Northbound API to program flow table rules on the switch for various use cases like L2 learning switch, Traffic Engineering, Firewall etc.			
4		Create a simple end-to-end network service with two VNFs using vim-emu https://github.com/containernet/vim-emu			
5		Install OSM and onboard and orchestrate network service.			

EX.NO : 1

DATE :

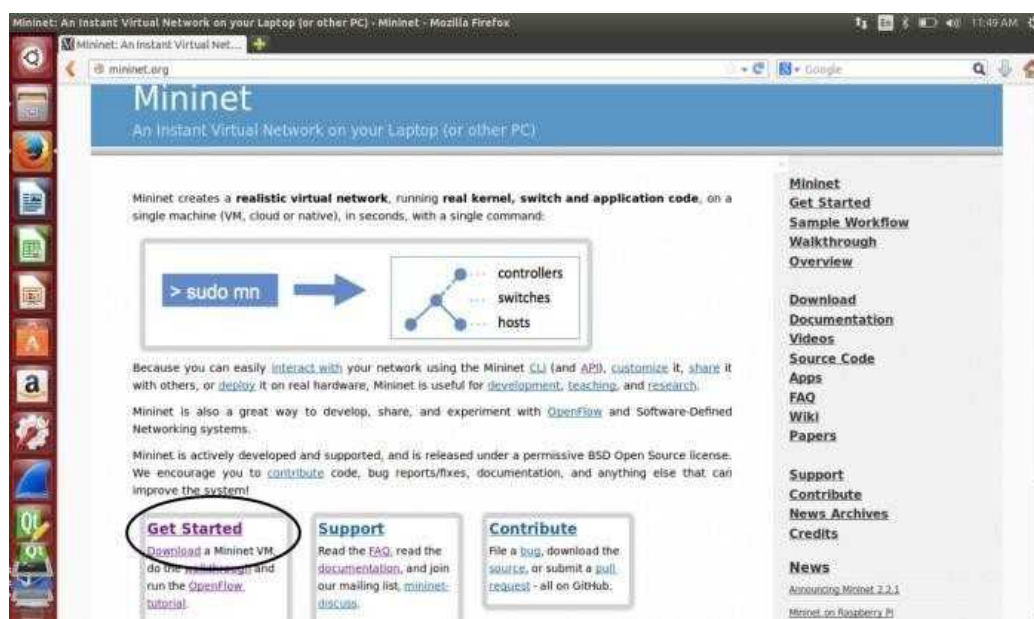
Setup your own virtual SDN lab Virtual Box /Mininet Environment for SDN

AIM:

To install virtual Box/Mininet environment for setup your own virtual Software definition network.

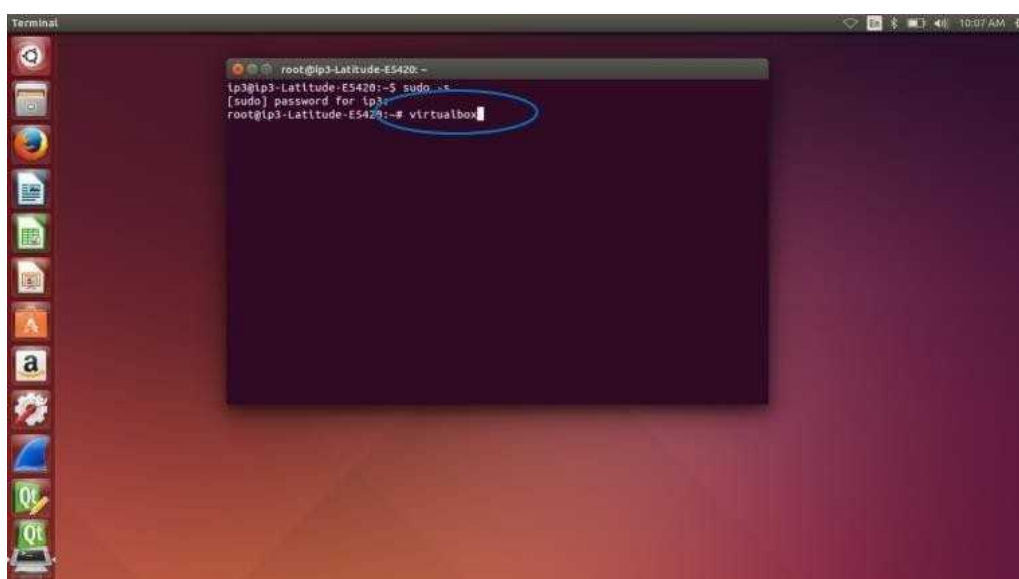
PROCEDURE:

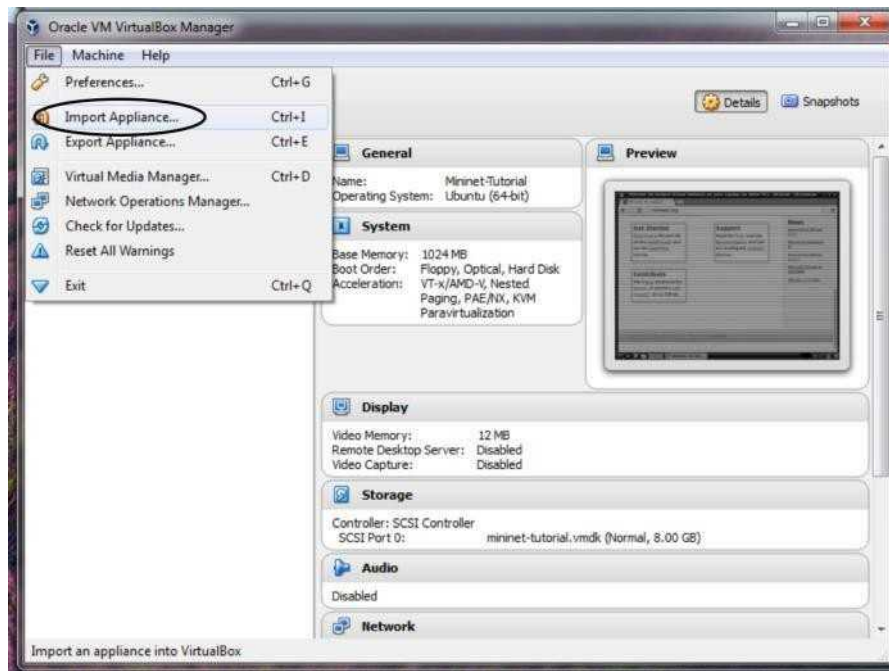
Step 1 : Download Mininet from <http://mininet.org> and start installation.



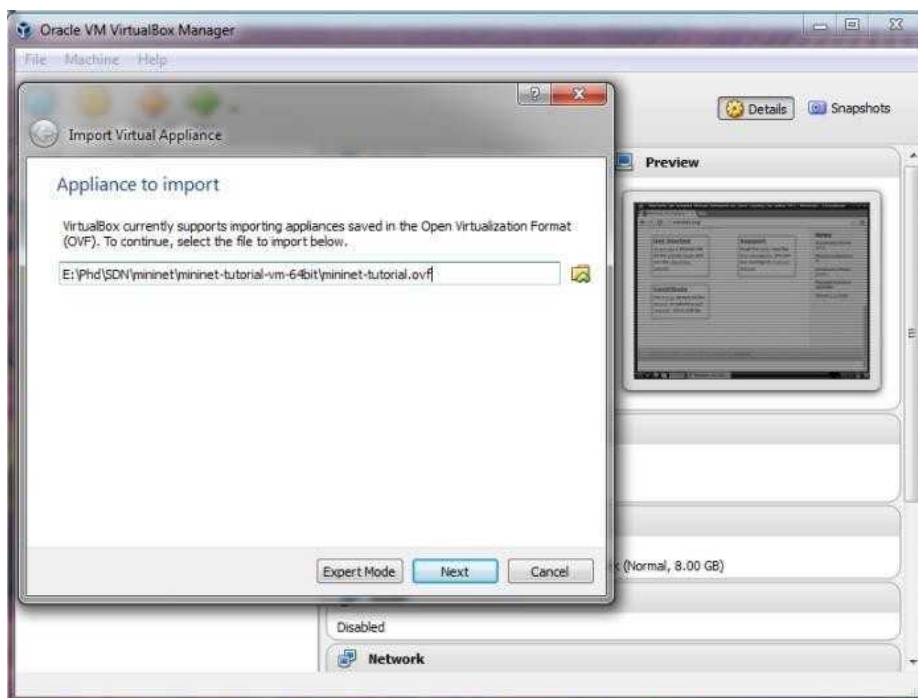
After installed mininet and Download VirtualBox click [here](#)

Step 2 : Then open oracle VM virtual box and choose File → Import Appliance.

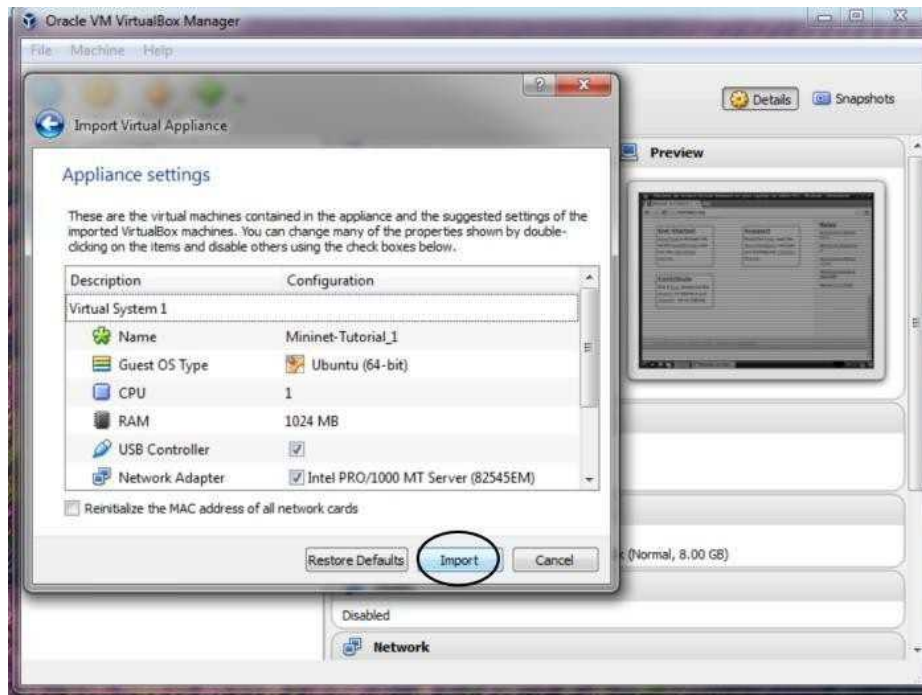




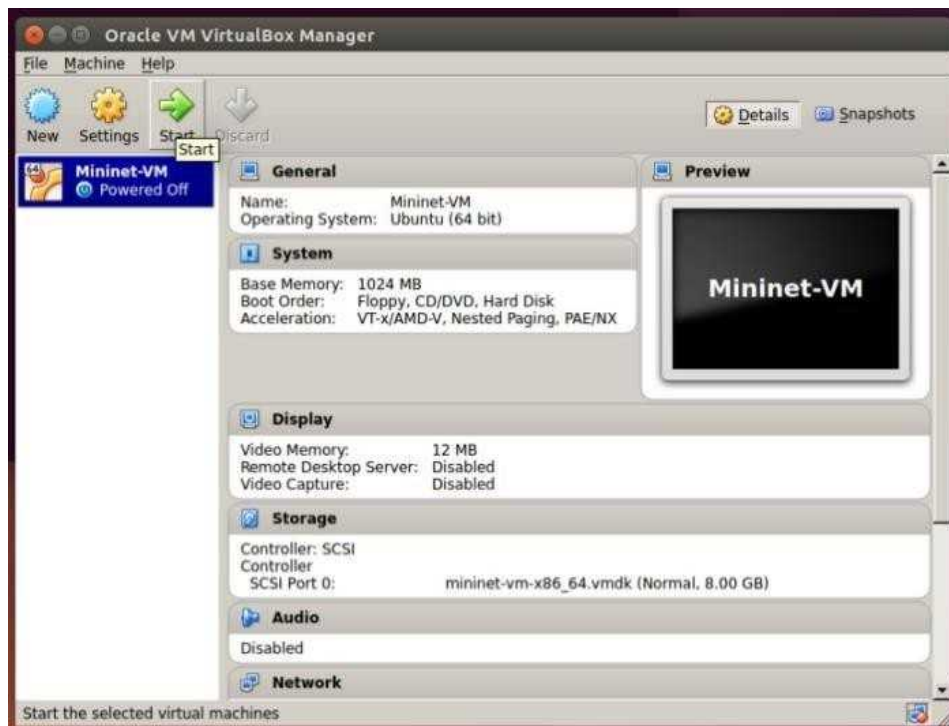
Step 3: Browse to open by clicking



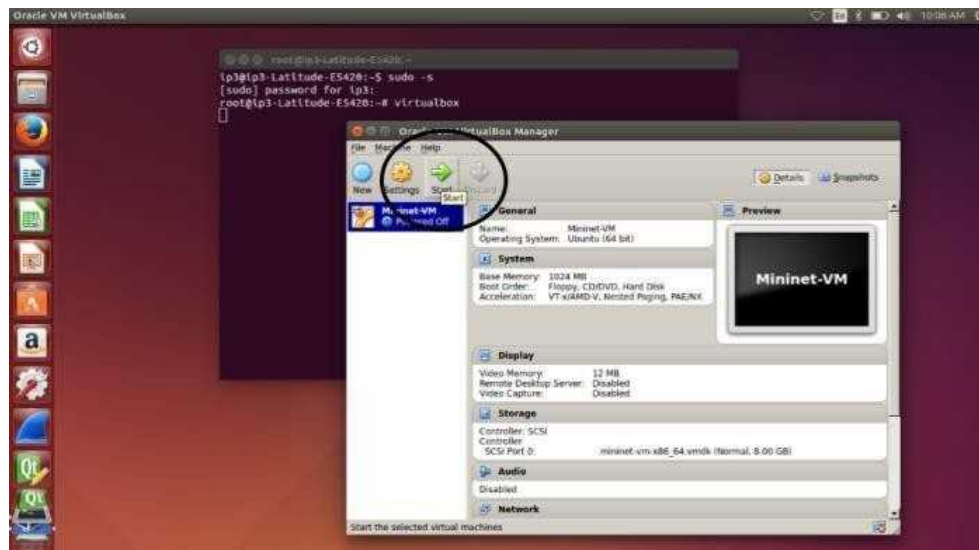
Step 4 : Click Next and Type Name as Mininet , select guest OS type as ubuntu 64bit or windows 2016 64bit ,CPU as 1 and RAM as 1024MB.



Step 5 : Click Import

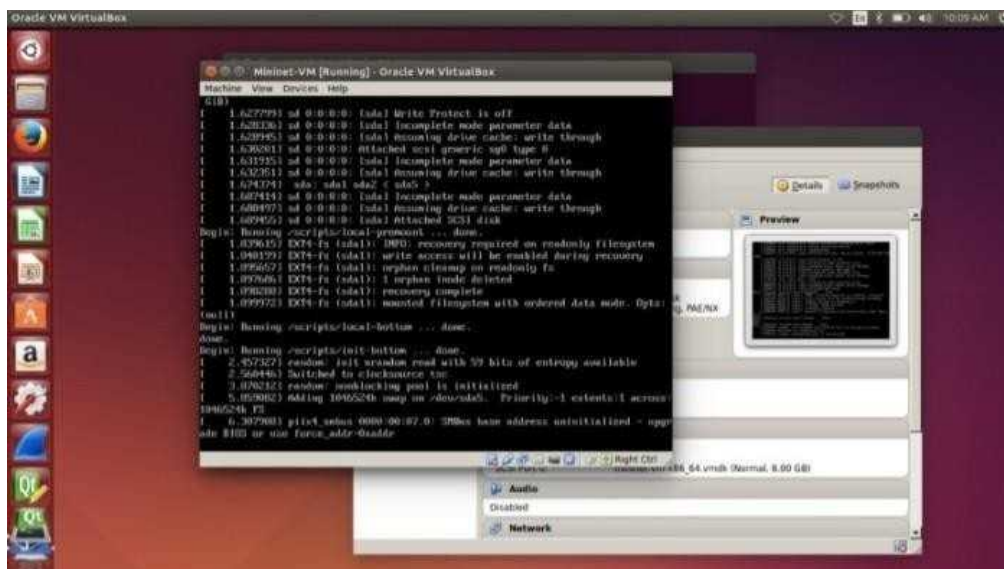


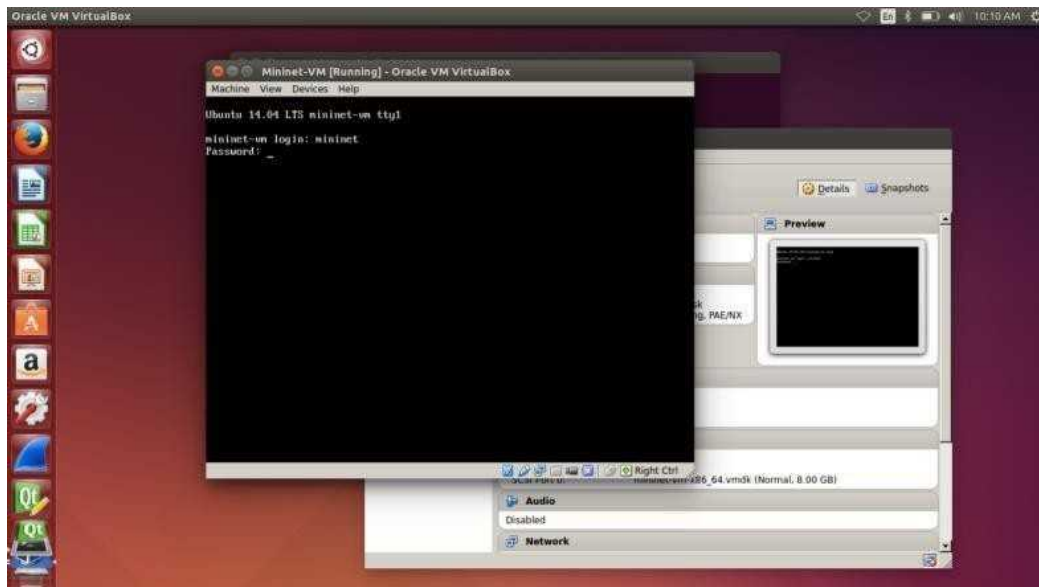
Step 6 : To Start Mininet. Click Start button to start a mininet...



Which takes some time to start to compile and updates of mininet in VM...

Username : mininet password: mininet



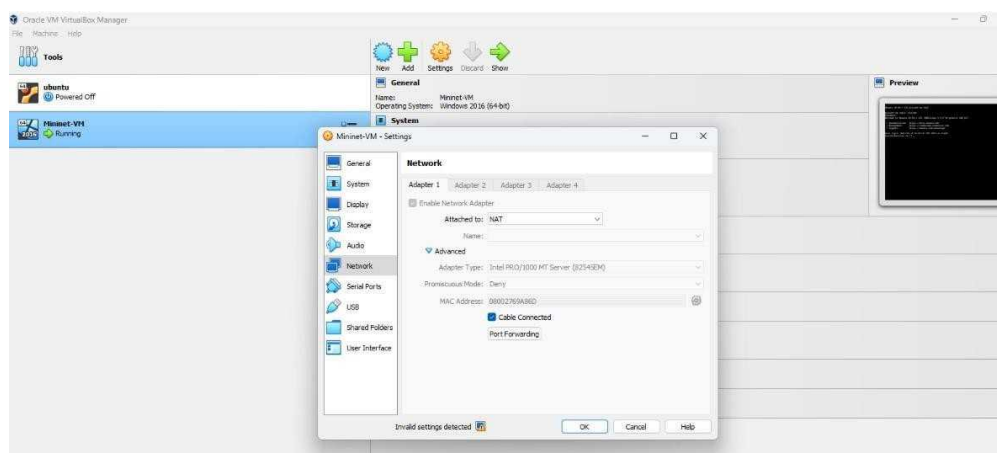


Step 7: After started mininet and again go to the settings then click the NEXT, after go to “attach to” select NAT option



Then advance to click the port forwarding. Then assign name=ssh, host port=2223,guest port=22 then click on OK.

Step 8: After this step open command prompt and Type cmd “>ssh Mininet@localhost -p 2223” & password: Mininet” too see the virtual network established in virtualbox.




```

Command Prompt
Microsoft Windows [Version 10.0.22621.3007]
(c) Microsoft Corporation. All rights reserved.

C:\Users\APCE54>ssh mininet@localhost -p 2223
mininet@localhost's password:
Welcome to Ubuntu 20.04.1 LTS (GNU/Linux 5.4.0-42-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

New release '22.04.3 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sat Feb  3 01:04:40 2024 from 10.0.2.2
mininet@mininet-vm:~$ ls
mininet  oflops  oftest  openflow  pox
mininet@mininet-vm:~$ cd mininet
mininet@mininet-vm:~/mininet$ ls
bin          custom  doc      INSTALL  Makefile  mn.1     mnexec.1  README.md  util
CONTRIBUTORS  debian  examples LICENSE  mininet   mnexec   mnexec.c  setup.py
mininet@mininet-vm:~/mininet$ exit
logout
Connection to localhost closed.

C:\Users\APCE54>

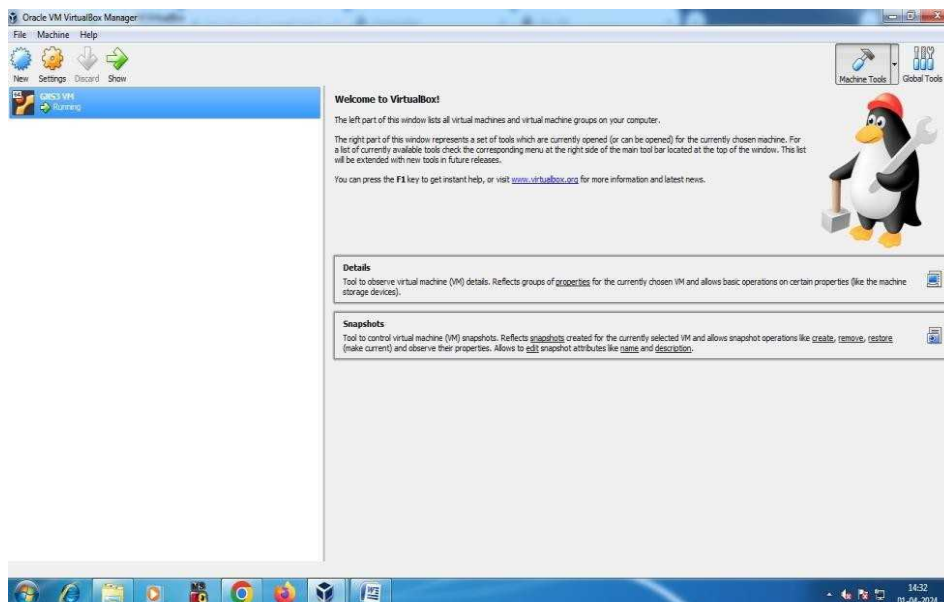
```

Type the commands \$ls to list the open APIs and again use \$cd Mininet to see the mininet environment and use \$exit command to exit from that network.

ii) <https://www.kathara.org>

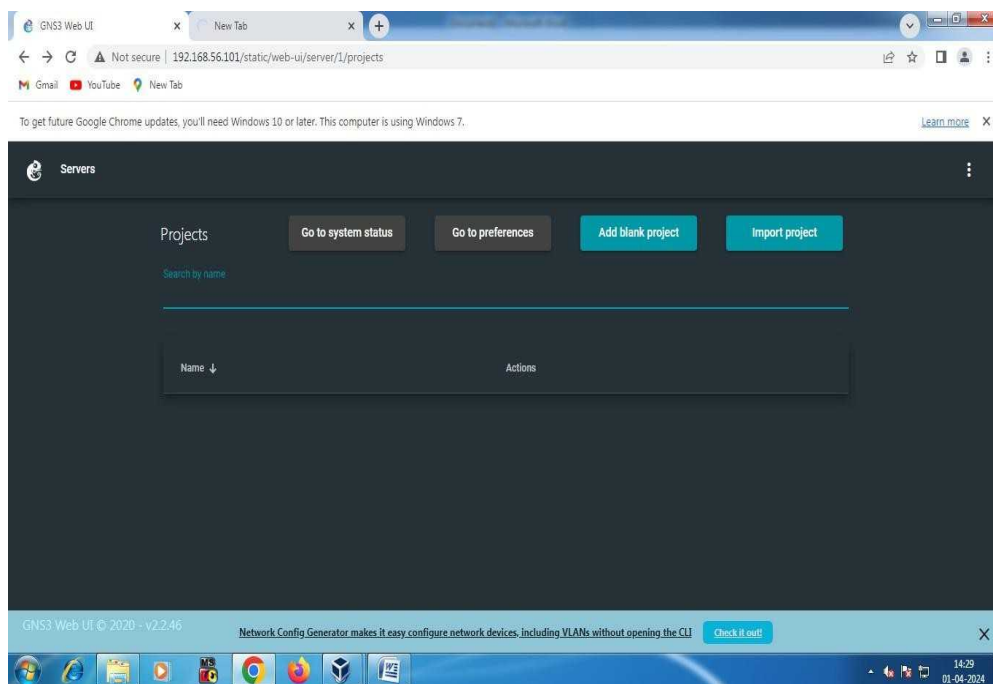
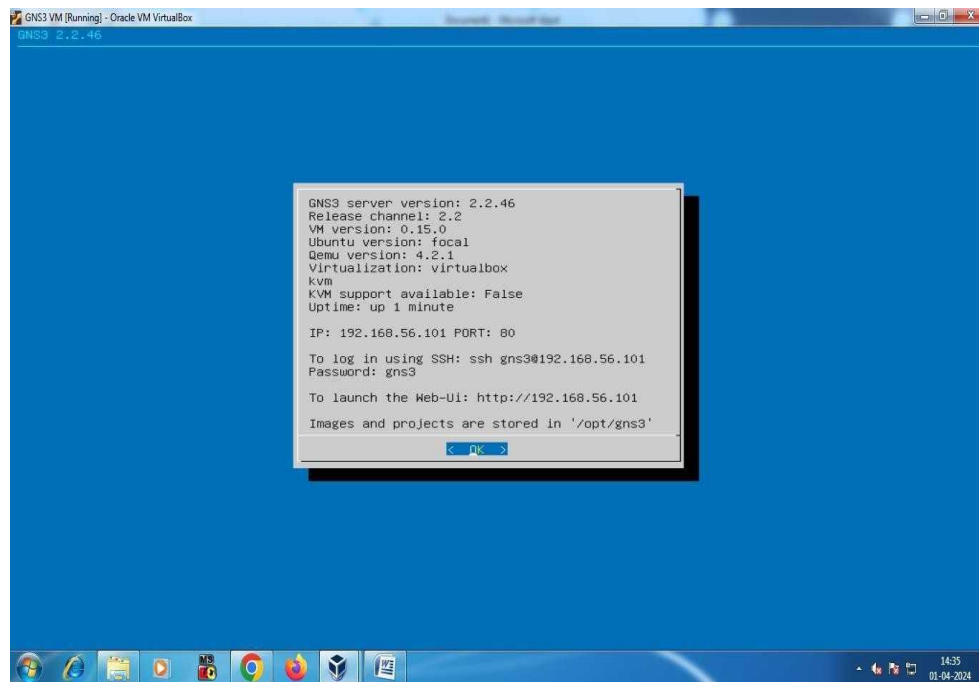
iii) GNS3

Step 1: Download GNS 3 software



Step 2: Extract the GNS3 file . then go to virtual box -> import appliance -> To import GNS 3.OVA

Step 3: GNS 3 loading process



RESULT:

Thus the above virtual box/mininet was successfully installed.

EX.NO : 2	Create a simple mininet topology with SDN controller and use Wireshark to capture and visualize the OpenFlow messages such as OpenFlow FLOW MOD, PACKET IN,PACKET OUT etc.
DATE :	

AIM:

1. Create a simple Mininet network with an SDN controller (Ryu).
2. Capture OpenFlow messages, including FLOW_MOD, PACKET_IN, and PACKET_OUT, using Wireshark.
3. Visualize the captured OpenFlow messages for analysis.

PROCEDURE:

Install Mininet:

Ensure you have Mininet installed on your system. You can use a Linux distribution for this task.

Install Ryu SDN Controller:

Install Ryu, a popular SDN controller, using pip: `pip install ryu`

Create the Mininet Topology:

Create a Python script (e.g., `mininet_topology.py`) to define your Mininet network topology and start Mininet with the Ryu controller. Here's an example topology with a single switch and two hosts:

```
python
from mininet.net import Mininet
from mininet.topo import SingleSwitchTopo
from mininet.node import RemoteController # Create a Mininet instance
net = Mininet(topo=SingleSwitchTopo(2), controller=RemoteController) # Start Mininet
net.start()
```

...

Start the Ryu Controller:

In a separate terminal, start the Ryu controller:

...

`ryu-manager`

...

Capture OpenFlow Messages with Wireshark:

Start Wireshark and select your network interface (e.g., `eth0`).

Apply a display filter to capture only OpenFlow messages. Use the filter expression: `of`.

Begin capturing packets by clicking the "Start" button in Wireshark.

Generate OpenFlow Messages:

In the Mininet terminal, you can use the Mininet CLI to generate OpenFlow messages. For example, you can add a flow rule (FLOW_MOD) or generate traffic (PACKET_OUT).

To add a flow rule (FLOW_MOD):

...

```
mininet> h1 ovs-ofctl add-flow s1 in_port=1,actions=output:2
```

...

To generate traffic (PACKET_OUT):

...

```
mininet> h1 ping -c 1 h2
```

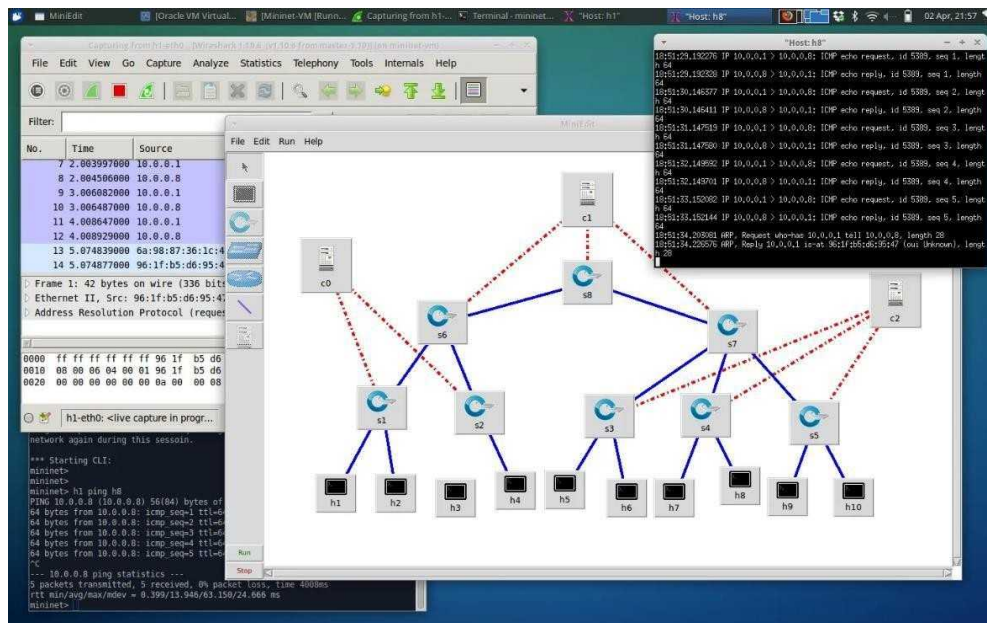
...

Stop Wireshark Capture:

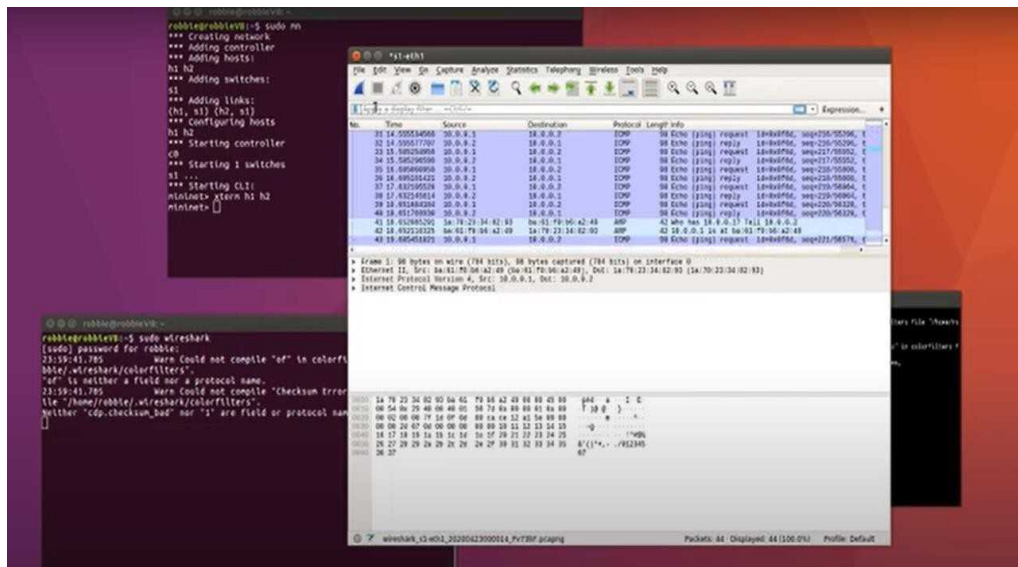
Stop capturing packets in Wireshark when you've generated enough OpenFlow messages.

OUTPUT:

The Wireshark capture should display OpenFlow messages exchanged between the Ryu controller and the Mininet switch. You will see messages like FLOW_MOD, PACKET_IN, and PACKET_OUT, along with their details.



Use Wireshark's visualization tools to analyze and inspect the captured OpenFlow messages. You can filter, sort, and drill down into specific messages to understand the communication between the controller and switches.



RESULT:

This setup allows you to observe and analyze the OpenFlow messaging in a simple SDN network. You can further customize the Mininet topology and generate more complex OpenFlow scenarios for testing and analysis.

EX.NO : 3	Create a SDN application that uses the Northbound API to program flow table rules on the , Traffic Engineering, Firewall etc
DATE :	

AIM:

To Develop an SDN application that uses the Northbound API to program flow table rules on SDN switches for different use cases: L2 learning switch, Traffic Engineering, and Firewall.

PROCEDURE:

Set Up the Development Environment:

Install the Ryu SDN controller and any necessary Python libraries.

Create the Ryu SDN Application:

Create a Python script for your Ryu SDN application, which will implement the Northbound API to program flow rules.

L2 Learning Switch Use Case:

In your Ryu application, use the Northbound API to program flow rules for basic L2 learning. For example, when a packet arrives, add a flow entry to the switch's flow table based on the source MAC address and port.

Traffic Engineering Use Case:

Implement traffic engineering rules using the Northbound API. For instance, you can define flow rules to optimize traffic paths or prioritize specific traffic based on application requirements.

Firewall Use Case:

Implement firewall rules using the Northbound API to drop or allow specific traffic based on criteria such as source/destination IP addresses, ports, or protocols.

Run the SDN Application:

Start your Ryu SDN application with the Ryu manager using the following command:

```

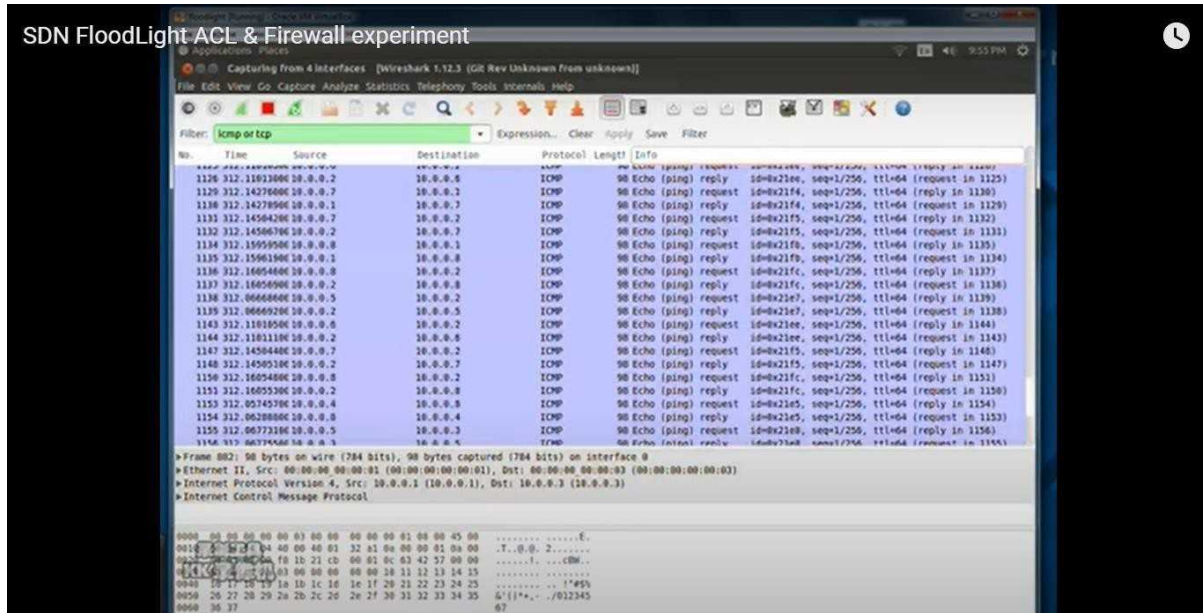
...
ryu-manager your_application.py
...
```

Test the SDN Application:

Create a virtual network or use a Mininet topology for testing your SDN application. Generate traffic to see how the application programs the flow table rules.

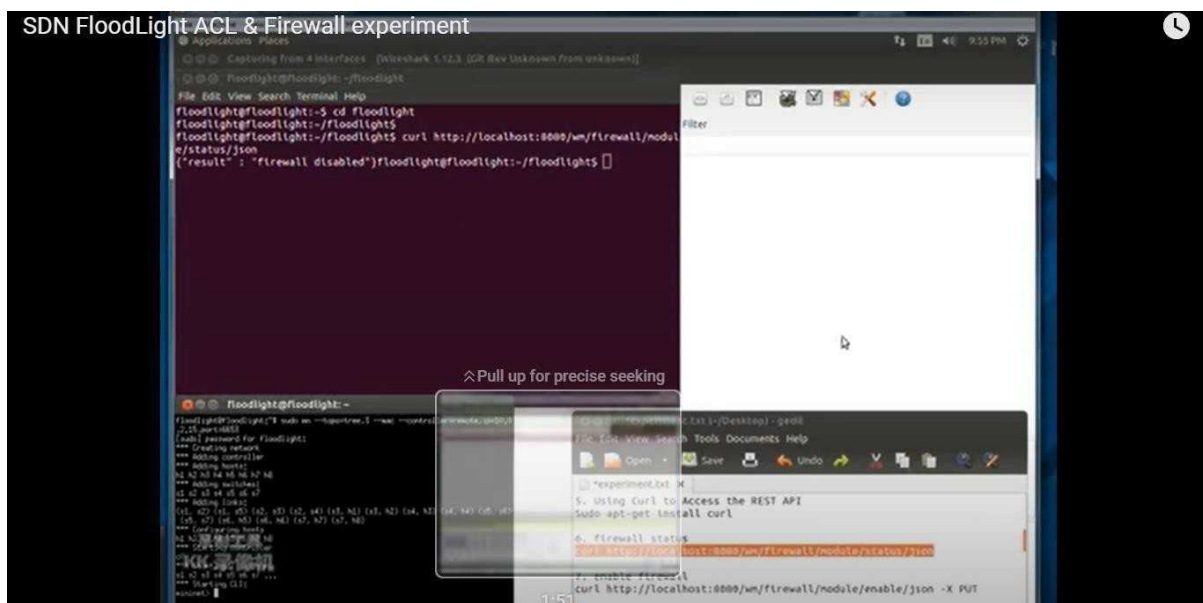
OUTPUT :

For the L2 learning switch use case, the application should program flow rules that enable the switch to learn and forward traffic based on MAC addresses.



In the traffic engineering use case, the application should optimize traffic paths or prioritize certain types of traffic as per your defined rules.

In the firewall use case, the application should allow or block traffic based on your defined criteria. The application's output should include log messages or other forms of feedback, showing that it is functioning correctly.



RESULT:

The result is a working SDN application that leverages the Northbound API to program flow table rules on SDN switches for different use cases. The application should effectively control network traffic based on the specified policies and rules.

EX.NO : 4	Create a simple end-to-end network service with two VNFs Using vim-emu https://github.com/containernet/vim-emu
DATE :	

AIM:

To create a simple end-to-end network service with two VNFs using vim-emu.

PROCEDURE:

- Set Up the Environment:

Make sure you have a Linux system. You can set up a virtual machine or a dedicated system.

- Install the required dependencies, including Docker and Docker Compose.

- Install vim-emu:

Follow the instructions in the vim-emu GitHub repository (<https://github.com/containernet/vim-emu>) to install vim-emu.

- Create Network Topology:

Define a network topology using vim-emu. You can create a Python script that specifies the network, VNFs, and their interconnections. For example, create a file named

`network_topology.py`:

```
```python
```

```
from mininet.net import Containernet
from mininet.node import Docker
from mininet.link import TCLink
from mininet.cli import CLI
net = Containernet()
```

```
vnf1 = net.addDocker('vnf1', dimage="vnf1_image")
vnf2 = net.addDocker('vnf2', dimage="vnf2_image")
net.addLink(vnf1, vnf2, cls=TCLink)
net.start()
CLI(net)
net.stop()
```

```
```
```

2. Create VNF Docker Images:

Build Docker images for your VNFs. Create a `Dockerfile` for each VNF, specifying its requirements and configurations. Build the images using Docker commands.

3. Launch the Network Topology:

Run your network topology with vim-emu:

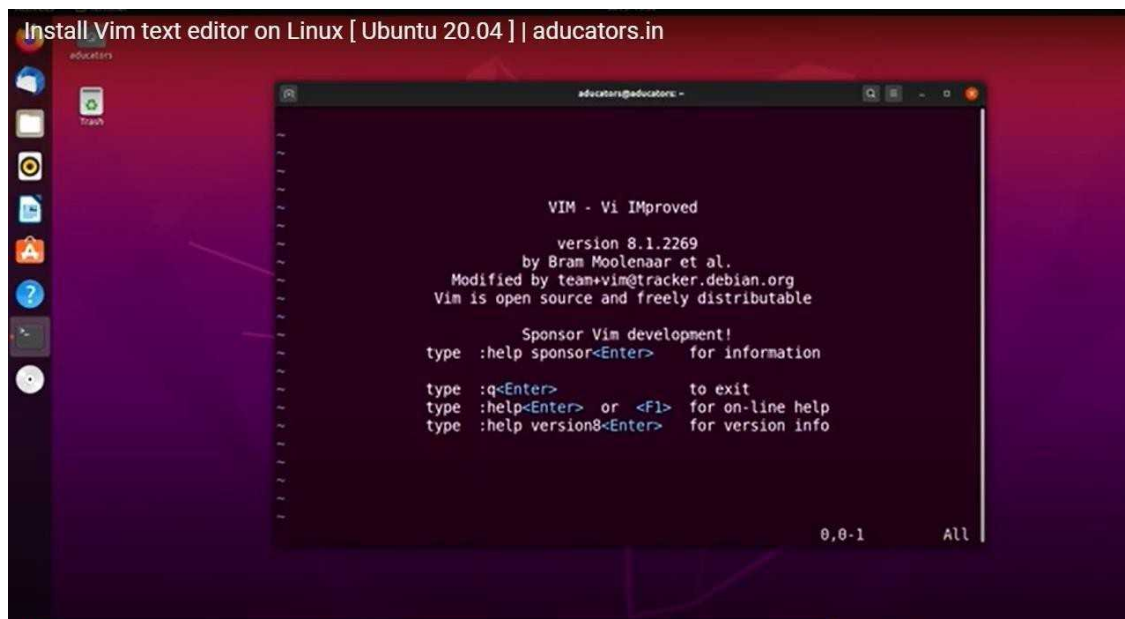
```
```
```

```
sudo python network_topology.py
```

```
```
```

OUTPUT:

- 
- aducators@educators:~\$ sudo apt install vim
- ```
Reading package lists... Done
Building dependency tree
Reading state information... Done
Suggested packages:
 ctags vim-doc vim-scripts
The following NEW packages will be installed:
 vim
0 upgraded, 1 newly installed, 0 to remove and 301 not upgraded.
Need to get 1,238 kB of archives.
After this operation, 3,111 kB of additional disk space will be used.
Get:1 http://in.archive.ubuntu.com/ubuntu focal/main amd64 vim amd64 2:8.1.2269-1ubuntu5 [1,238 kB]
Fetched 1,238 kB in 0s (8,203 kB/s)
```



The result is a fully orchestrated end-to-end network service with two VNFs, established using vim-emu and OSM. The network service is now ready to accept and process traffic as specified in your descriptor.

<b>EX.NO : 5</b>	<b>Install OSM and onboard and orchestration network service</b>
<b>DATE :</b>	

**AIM:**

To Install OSM and onboard and orchestrate the network service.

**PROCEDURE:**

## 1. Install OSM:

Follow the instructions on the OSM GitHub repository (<https://osm.etsi.org>) to install OSM, which includes installing the OSM client and server components.

## 2. Onboard the Network Service:

Use the OSM client to onboard your network service. You'll need to provide a descriptor (e.g., a TOSCA YAML file) for your service.

...

**osm ns-create <your\_network\_service\_descriptor\_file>**

...

## 3. Instantiate the Network Service:

Instantiate the network service using OSM:

...

**osm ns-instantiate <ns-instance-name> <your\_network\_service\_name>**

...

Replace ``<ns-instance-name>`` with a suitable name for your instantiated service.

**OUTPUT:**

- After installing OSM and onboarding the network service descriptor, you should see your network service listed within OSM.
- When instantiating the network service, OSM will orchestrate the deployment of the VNFs and connect them according to your descriptor.

**RESULT:**

Thus the result of install OSM and onboard and orchestrate the network service. The network service is now ready to accept and process traffic as specified in your descriptor.